

Spark 고급과 Spark ML

Shuffling시 Skew 처리방식과 Spark ML에 대해
배워보자

Grepp, Inc

한기용

keeyonghan@hotmail.com

Contents

1. Spark 기타 기능과 메모리 관리
2. Spark Shuffling 최적화
3. Spark Partition 학습
4. Spark ML 소개와 ML 모델 빌딩
5. ML Pipeline과 Tuning 소개와 실습

Spark 기타 기능과 메모리 관리

기타 기능과 메모리 관리에 대해 알아보자

Grepp, Inc

한기용

keeyonghan@hotmail.com

Contents

1. 기타 기능/개념 살펴보기
2. Driver와 Executor 해부
3. 메모리 이슈 정리
4. JVM과 Python 간의 통신
5. Caching과 Persist
6. Dynamic Partition Pruning

◆ 살펴볼 기능과 개념

- ❖ Broadcast Variable
- ❖ Accumulators
- ❖ Speculative Execution
- ❖ Scheduler
- ❖ Dynamic Resource Allocation

◆ Broadcast Variable이란 무엇인가?

- ❖ 룩업 테이블등을 브로드캐스팅하여 셔플링을 막는 방식으로 사용
 - 브로드캐스트 조인에서 사용되는 것과 동일한テクニック
 - 대부분 룩업 테이블 (혹은 디멘션 테이블 - 10-20MB)을 **Executor**로 전송하는데 사용
 - 많은 DB에서 스타 스키마 형태로 팩트 테이블과 디멘션 테이블을 분리
 - `spark.sparkContext.broadcast`를 사용

◆ Broadcast Variable: 록업 테이블(파일)을 UDF로 보내는 방법

❖ Closure

- Serialization이 태스크 단위로 일어남
- UDF안에서 파이썬 데이터 구조를 사용하는 경우

❖ Broadcast

- Serialization이 **Worker Node** 단위로 일어남 (그 안에서 캐싱되기에 훨씬 더 효율적)
- UDF안에서 브로드캐스트된 데이터 구조를 사용하는 경우

❖ Broadcast 데이터셋의 특징

- **Worker node**로 공유되는 변경 불가 데이터
- **Worker node**별로 한번 공유되고 캐싱됨
- 제약점은 **Task Memory**안에 들어갈 수 있어야함

◆ Broadcast Variable: 예제

- ❖ 가장 인기있는 마벨 슈퍼히로를 찾는 예제를 보자
- ❖ 슈퍼히로의 **ID**를 찾고 그에 해당하는 이름을 찾아야함
 - 이때 룩업 테이블을 **DataFrame**으로 로딩하고 조인을 하는 것도 방법
 - 아니면 룩업 테이블을 브로드캐스트하여 **UDF**안에서 사용하는 것도 방법
- ❖ 예제코드

id connections	
5988	48
5989	40
5982	42
5983	14
5980	24

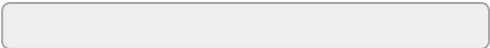
id name	
1	24-HOUR MAN/EMMANUEL
2	3-D MAN/CHARLES CHAN
3	4-D MAN/MERCURIO
4	8-BALL/
5	A

◆ Accumulators란?

- ❖ 특정 이벤트의 수를 기록하는데 사용됨 -> 일종의 전역 변수
 - 하둡에서 카운터와 아주 흡사
- ❖ 예를 들면 비정상적인 값을 갖는 레코드의 수를 세는데 사용

State	City	Zipcode
California	Sunnyvale	9511
California	Mountain View	94111
California	Cupertino	94123
California	San Jose	951

5자리가 아닌
Zipcode는
NULL로 변경

```
def handle_bad_zipcode(c: int) -> int:  
    if len(str(c)) != 5:  
          
        return None  
    return c
```

◆ Accumulators의 특징

- ❖ 변경 가능한 전역변수로 드라이버에 위치
- ❖ 스칼라로 만들면 이름을 줄 수 있지만 그 이외에는 불가
 - 이름있는 accumulator만 Spark Web UI에 나타남
- ❖ 레코드 별로 세거나 합을 구하는데 사용 가능
- ❖ 두 가지 방법으로 사용 가능하며 값의 정확도도 달라짐
 - Transformation에서 사용
 - 이 경우 값이 부정확할 수 있음 (태스크의 재실행과 speculative execution)
 - DataFrame/RDD Foreach에서 사용
 - 추천되는 방식으로 이 경우 정확함
- ❖ 예제코드

◆ Speculative Execution란?

- ❖ 느린 태스크를 다른 Worker node에 있는 Executor에서 중복 실행
 - 이를 통해 Worker node의 하드웨어 이슈등으로 느려지는 경우 빠른 실행을 보장
 - 하지만 Data Skew로 인해 오래 걸린다면 도움이 안되고 리소스만 낭비하게 됨



◆ Speculative Execution 제어 방식

- ❖ spark.speculation으로 컨트롤 가능하며 기본은 **False** (비활성화)
 - 하둡 MapReduce에서부터 있던 기능
- ❖ 다양한 환경변수로 세밀하게 제어 가능

환경 변수 이름	Default	LinkedIn
spark.speculation.interval	100ms	1 sec
spark.speculation.multiplier	1.5	4
spark.speculation.quantile	0.75	0.9
spark.speculation.minTaskRuntime	100ms	30 sec
spark.speculation.task.duration.threshold	None	None

◆ Spark의 리소스 할당 (스케줄링)

❖ Spark Application들간의 리소스 할당

- 기반이 되는 리소스 매니저가 결정
 - YARN은 세 가지 방식 지원: FIFO, FAIR, CAPACITY
- 한번 리소스를 할당받으면 해당 리소스를 끝까지 들고 가는 것이 기본

❖ 하나의 Spark Application안에서 잡들간의 리소스 할당

- FIFO 형태로 처음 잡이 필요한대로 리소스를 받아서 쓰는 것이 기본

◆ Spark Application의 리소스 요구/릴리스 방식

❖ Static Allocation (기본 동작)

- Spark Application은 리소스 매니저로부터 (YARN) 받은 리소스를 보통 끝까지 들고감
- 이는 리소스 사용률에 악영향을 줄 가능성이 높음

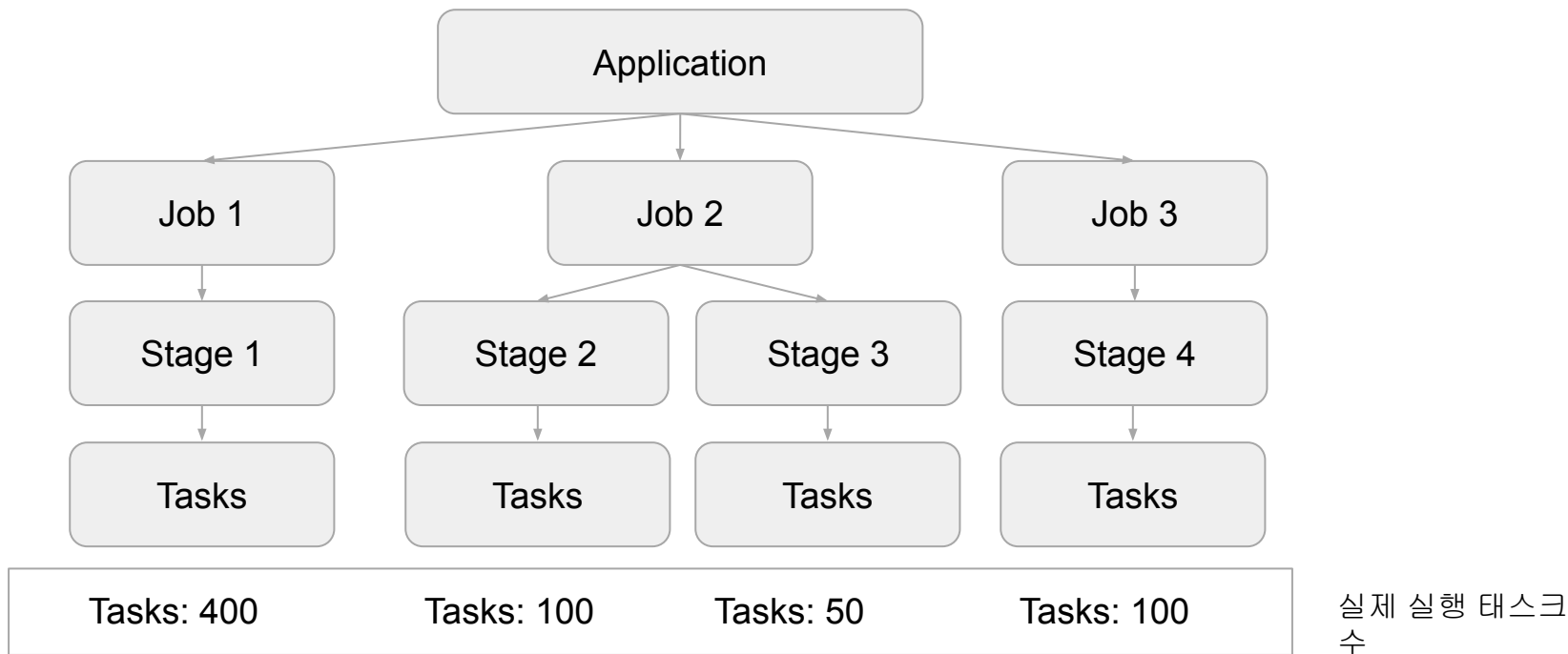
❖ Dynamic Allocation

- Spark Application이 상황에 따라 **executor**를 릴리스하기도 하고 요구하기도 함
- 다수의 Spark Application들이 하나의 리소스 매니저를 공유한다면 활성화하는 것이 좋음

```
spark-submit --num-executors 100 --executor-cores 4 --executor-memory 32G
```

◆ Static Allocation vs. Dynamic Allocation

```
spark-submit --num-executors 100 --executor-cores 4 --executor-memory 32G
```



◆ Dynamic Resource Allocation

❖ Dynamic Resource Allocation은 아래 환경변수들로 제어

`spark.dynamicAllocation.enabled = true`

`spark.dynamicAllocation.shuffleTracking.enabled = true`

`spark.dynamicAllocation.executorIdleTimeout = 60s` (릴리스 타이밍 결정)

`spark.dynamicAllocation.schedulerBacklogTimeout = 1s` (요청 타이밍 결정)

`spark.dynamicAllocation.minExecutors`

`spark.dynamicAllocation.maxExecutors`

`spark.dynamicAllocation.initialExecutors`

`spark.dynamicAllocation.executorAllocationRatio`

<https://spark.apache.org/docs/latest/configuration.html#dynamic-allocation>

◆ Spark Scheduler란?

❖ 하나의 Spark Application내의 잡들에 리소스를 나눠주는 정책

- Spark Application들간에 리소스를 나눠주는 방식은 리소스 매니저에게 달려있음

❖ 다음 2 가지가 존재

- “FIFO” (기본)
 - 리소스를 처음 요청한 Job에게 리소스 우선 순위가 감
- “FAIR”
 - 라운드로빈 방식으로 모든 잡에게 고르게 리소스를 분배하는 방식
 - 이 안에서 풀(Pool)이란 형태로 리소스를 나눠서 우선순위 고려한 형태로 사용 가능
 - 풀안에서 리소스 분배도 FAIR 혹은 FIFO로 지정 가능

◆ Scheduler를 활용한 병렬성 증대

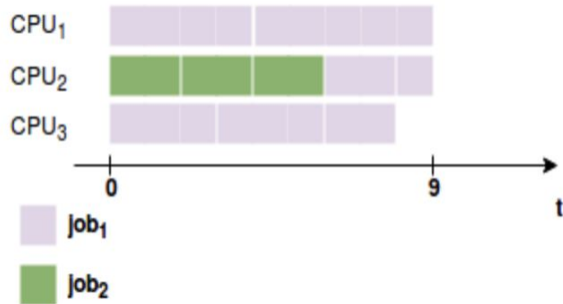
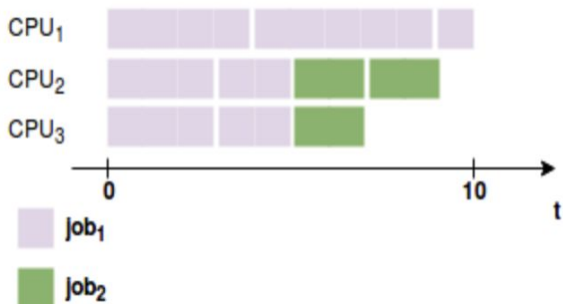
❖ 병렬성 증대 -> Thread 활용이 필요

- 이는 FAIR 모드의 스케줄러일 경우 더 효과적

❖ 관련 환경 변수

- `spark.scheduler.mode`: FIFO (기본) 혹은 FAIR
- `spark.scheduler.allocation.file`: “FAIR”은 경우 필요하며 풀을 정의해놓는 형태로 사용됨

❖ 스케줄러 데모 (스레드 사용)



Driver와 Executor 해부

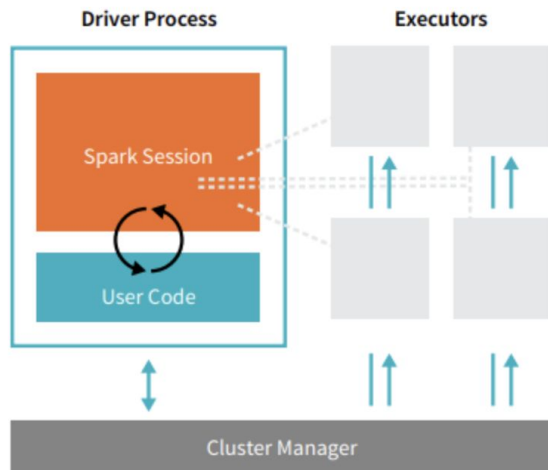
Driver와 Executor의 리소스 사용에 대해 알아보자

◆ Driver의 역할

❖ Spark Application = (1 Driver) + (1+ Executor)

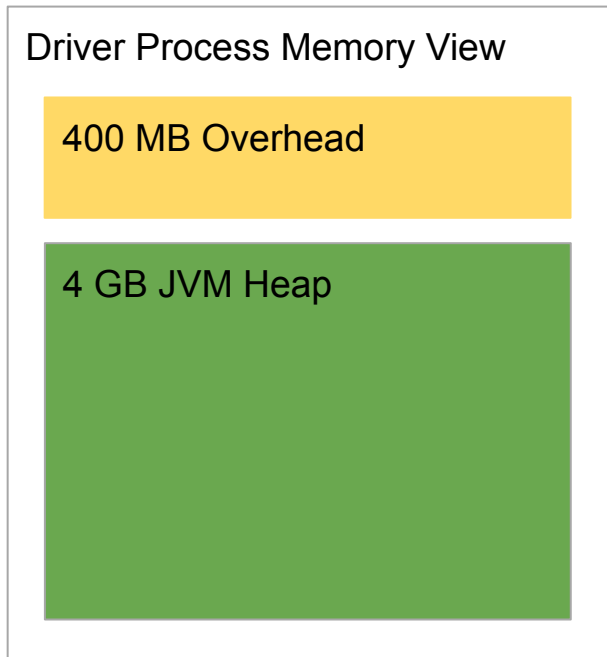
❖ Driver는 다음 역할을 수행

- main 함수 실행하고 SparkSession/SparkContext를 생성
- 코드를 태스크로 변환하여 DAG 생성
- 이를 execution/logical/physical plan으로 변환
- 리소스 매니저의 도움을 받아 태스크들을 실행하고 관
 - task의 수가 너무 많아지면 driver 메모리 에러 발생
- 위의 정보들을 Web UI로 노출시킴 (4040 포트)



Source: Databricks

◆ Driver 메모리 구성



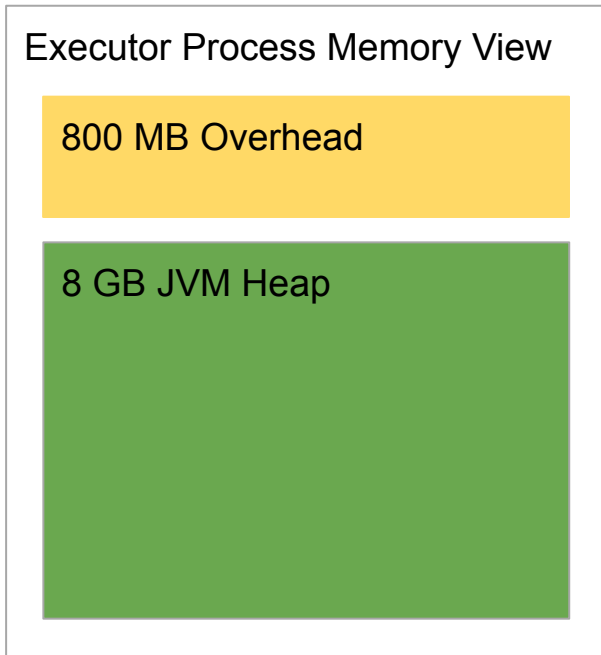
spark.driver.memory = 4GB

spark.driver.cores = 4

spark.driver.memoryOverhead = 0.1

- $\max(\text{spark.driver.memory의 } 10\%, 384\text{MB})$

◆ Executor 메모리 구성



spark.executor.memory = 8GB

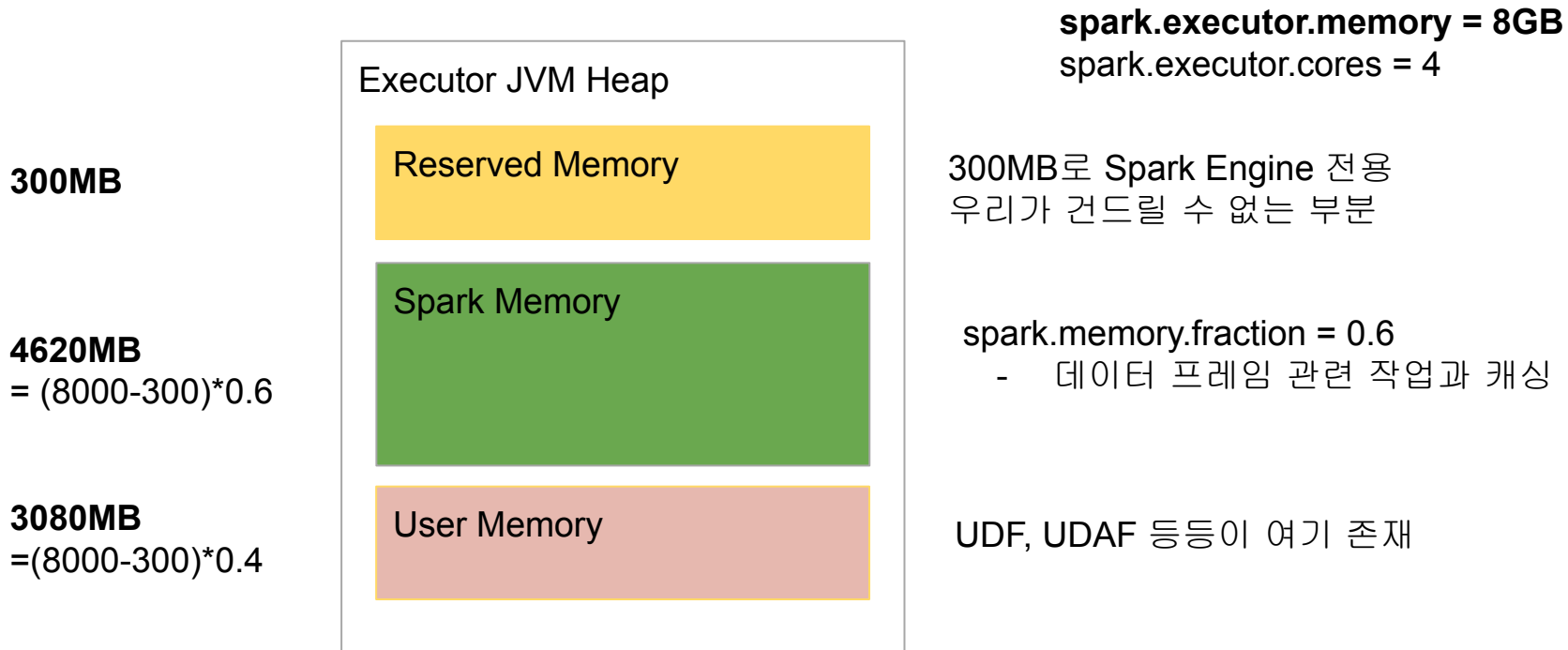
spark.executor.cores = 4

spark.executor.memoryOverhead = 0.1

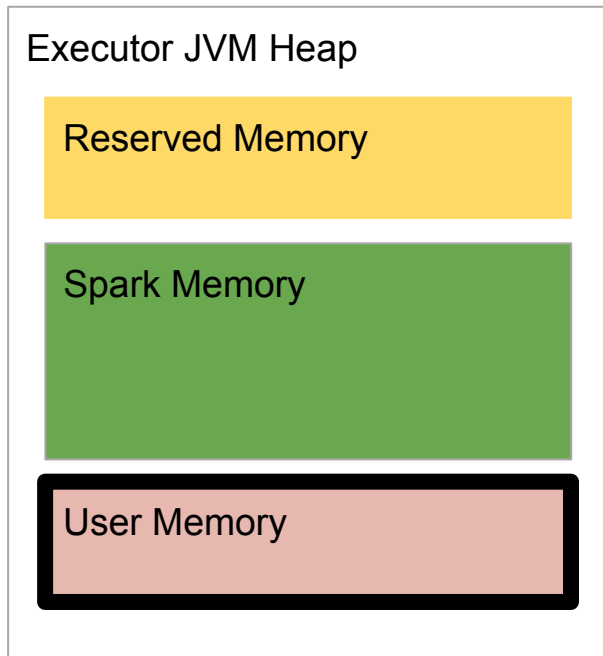
- max(spark.executor.memory의 10%, 384MB)

yarn.nodemanager.resource.memory-mb

◆ 메모리 구성 - Heap 메모리(8GB)만 보자



◆ 메모리 구성 - User Memory를 보자



spark.executor.memory = 8GB

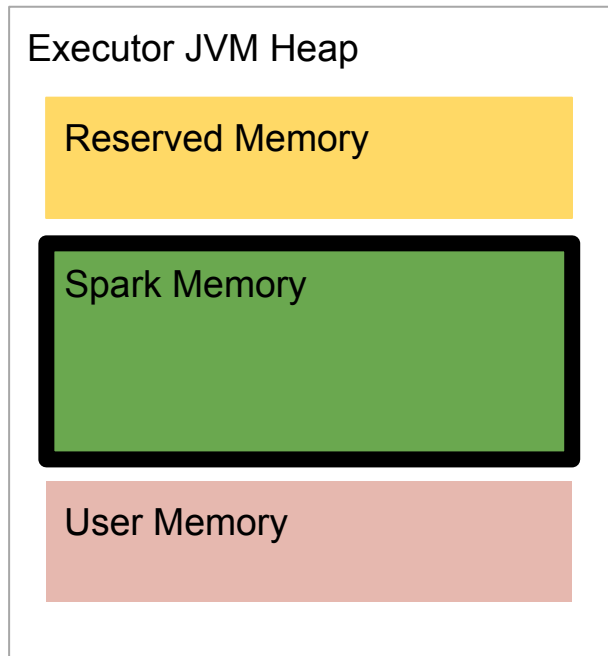
spark.executor.cores = 4

남은 메모리 (spark memory와 reserved memory 제외)

- User-defined data structure
- Spark internal metadata
- UDF
- RDD conversion operation
- RDD lineage and dependency

- RDD와 관련된 작업을 직접 하는 경우 사용
- 이 부분이 적다면 `spark.memory.fraction`을 증가

◆ 메모리 구성 - Spark Memory를 보자



spark.executor.memory = 8GB

spark.executor.cores = 4

Storage Memory Pool
- DataFrame Caching

Executor Memory Pool
- DataFrame
Operation

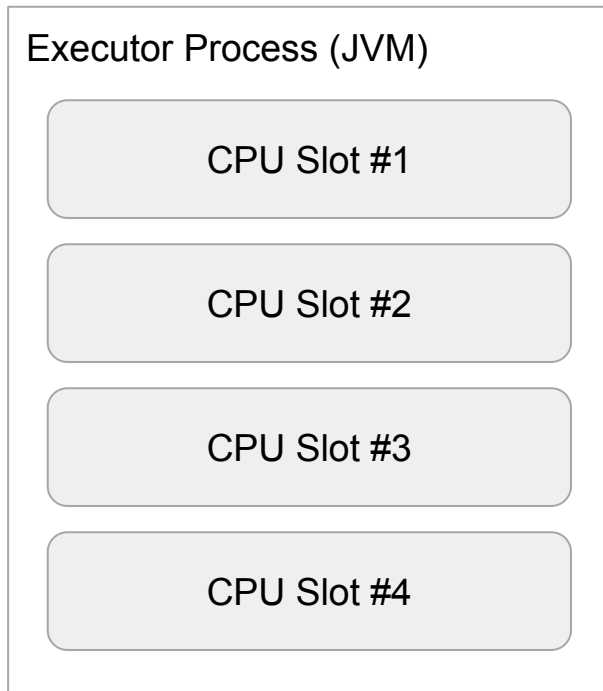
spark.memory.storageFraction = 0.5

Off Heap Memory

PySpark Memory

Py4J Memory

◆ Executor CPU



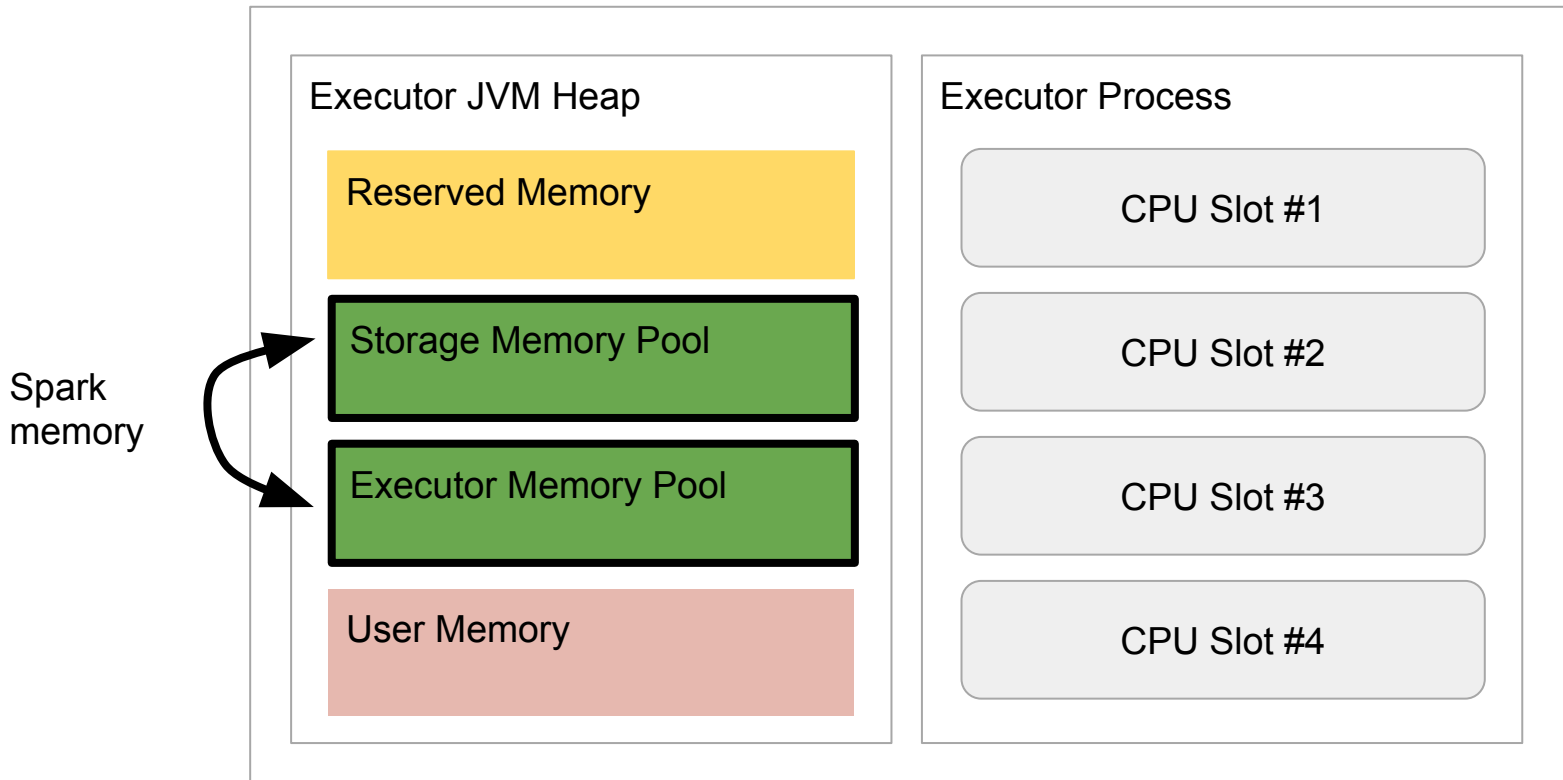
`spark.executor.memory = 8GB`
`spark.executor.cores = 4`

4 태스크가 병렬로 실행 가능함

Slot들은 사실 같은 JVM안의 thread를
말함

◆ Executor Resource

4개의 슬롯은 JVM 힙 메모리(특히 **Executor Memory Pool**)를 어떻게 공유할까?



◆ Executor Memory Pool Management

❖ Static Memory Management

- Spark 1.6전에는 슬롯들끼리 공평하게 나눠 가짐

❖ 지금은 Unified Memory Manager를 사용함

- 동작중인 태스크 대상으로 Fair Allocation이 기본 동작
 - 즉 실행중인 태스크가 모든 메모리를 가져가는 구조임

◆ Unified Memory Management

❖ Executor 메모리가 부족해지기 시작하면 어떤 일이 생길까?

- 메모리가 부족해지면 **Storage Memory Pool**에 남는 메모리를 사용
- `spark.memory.storageFraction`로 지정된 비율은 시작 비율
 - 하지만 양쪽의 메모리가 차기 시작하면 이 경계선은 지켜지면서 [eviction](#) 발생

❖ 반대로 **Storage** 메모리가 부족하기 시작한다면?

- **DataFrame** 캐싱을 하기위한 메모리가 부족해지면 **Executor Memory Pool**에 남는 메모리 사용
- 하지만 결국 `spark.memory.storageFraction` 이상은 넘어가지지 않음

❖ 더 이상 쓸 메모리가 없다면?

- 데이터를 메모리에서 디스크로 옮김 (**disk spill**)
- 디스크로 **spill**을 할 수 없다면 **OOM (Out of Memory)** 발생

◆ Spark Executor Memory Configuration (1)

이름	기본값	설명
spark.driver.memory	1G	
spark.driver.cores	1	
spark.executor.memoryOverhead	executorMemory * spark.executor.memoryOverheadFactor 와 384 중 더 큰 값 (MB)	Non-JVM 작업에 부여되는 메모리
spark.executor.memory	1G	JVM에 부여되는 메모리
spark.memory.fraction	0.6	Spark 메모리: JVM에 부여되는 메모리 중 데이터프레임 관련 메모리 비율 (캐싱 포함)
spark.memory.storageFraction	0.5	Spark 메모리 중 캐싱에 사용되는 메모리의 비율
spark.executor.cores	1 (YARN)	보통 executor 당 하나에서 5개의 CPU를 할당하는 것이 일반적

◆ Spark Executor Memory Configuration (2)

이름	기본값	설명
spark.memory.offHeap.enabled	False	Non-JVM 작업에 부여되는 별도 메모리 활성화 여부
spark.memory.offHeap.size	0	위의 offHeap에 부여되는 메모리
spark.executor.pyspark.memory	없음	파이썬 프로세스에 지정되는 메모리. 지정되지 않으면 Non-JVM 메모리(오버헤드 메모리)를 사용. 지정되면 이 값만 사용
spark.python.worker.memory	512	Py4J에 지정되는 메모리 (MB)

결국 하나의 **Executor**에 할당되는 메모리는 아래와 같음:

`spark.executor.memoryOverhead + spark.executor.memory + spark.memory.offHeap.size + spark.executor.pyspark.memory (+ spark.python.worker.memory)`

◆ What is Off Heap Memory?

❖ Spark은 On Heap 메모리에서 가장 잘 동작함

- 하지만 JVM Heap은 Garbage collection의 대상이 됨
- JVM Heap의 크기가 클수록 Garbe collection으로 인한 비용 증가
- 이 때 같이 사용할 수 있는 것이 JVM 밖에 있는 메모리
 - Overhead 메모리
 - Off Heap 메모리
 - spark.memory.offHeap.enabled를 true로 설정
 - spark.memory.offHeap.size에 원하는 크기 지정

◆ Spark 3.x의 Off heap memory 정리

❖ Spark 3.x는 Off Heap memory 작업에 최적화

- JVM 없이 직접 메모리 관리 가능 ([프로젝트 텅스톤](#))

❖ Spark 3.x는 Off Heap 메모리를 DataFrame용으로 사용

- GC의 발생을 줄일 수 있음

❖ 즉 Off Heap 메모리의 크기는

- `spark.executor.memoryOverhead + spark.offHeap.size`
- `spark.offHeap.size`를 사용해서 executor memory의 증가없이 off heap 메모리 증가가 가능



메모리 이슈 정리

Driver와 Executor에서 발생가능한 메모리 이슈들을
정리해보자

◆ Spark 메모리 이슈 (OOM)

- ❖ Driver OOM

- ❖ Executor OOM

```
java.lang.OutOfMemoryError: Java heap space
    at java.base/java.util.Arrays.copyOf(Arrays.java:3745)
    at java.base/java.io.ByteArrayOutputStream.grow(ByteArrayOutputStream.java:120)
    at java.base/java.io.ByteArrayOutputStream.ensureCapacity(ByteArrayOutputStream.java:95)
```

◆ Driver OOM 케이스들

- ❖ 큰 데이터셋에 collect 실행
- ❖ 큰 데이터셋을 Broadcast JOIN
- ❖ Python이나 R 등으로 작성된 코드
- ❖ 너무 많은 태스크들

◆ Executor OOM 케이스들

❖ 너무 큰 executor.cores 값

- High Concurrency

❖ Data Skew (Big Partition)

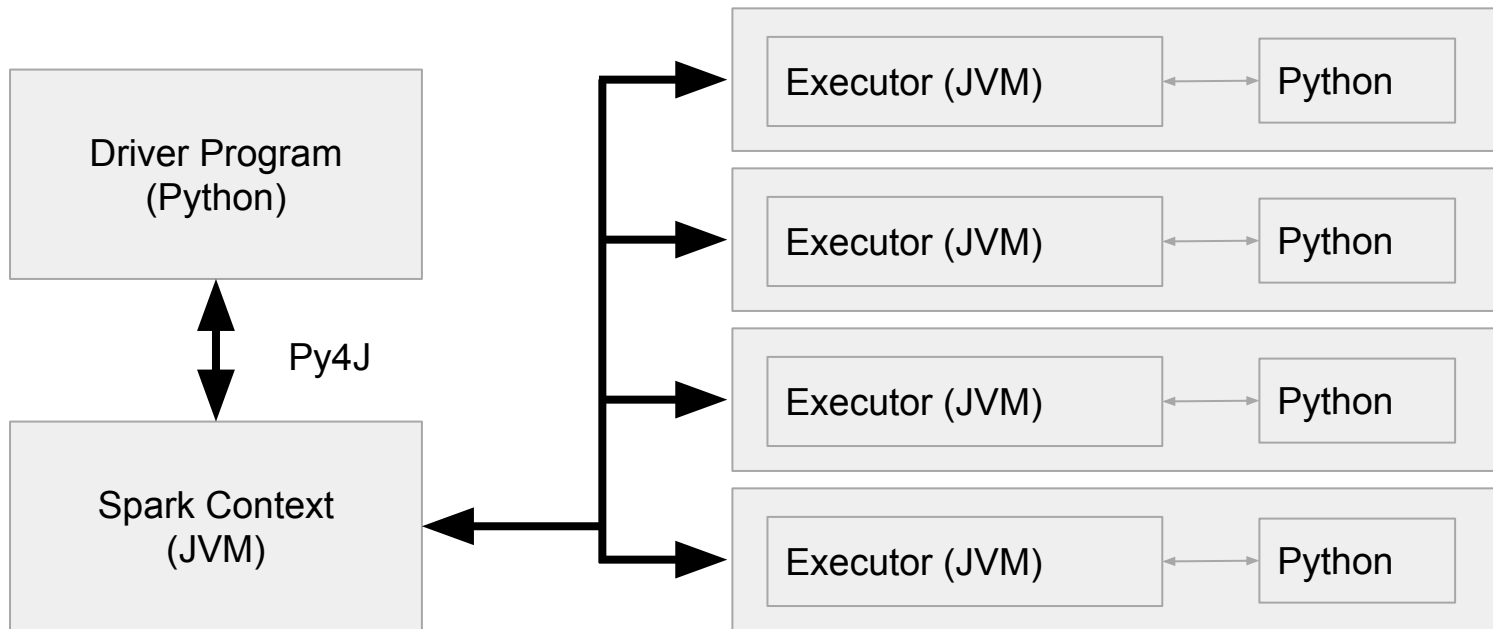


JVM과 Python 간의 통신

JVM과 Python 프로세스들간의 통신에 대해서 알아보자

◆ PySpark Driver

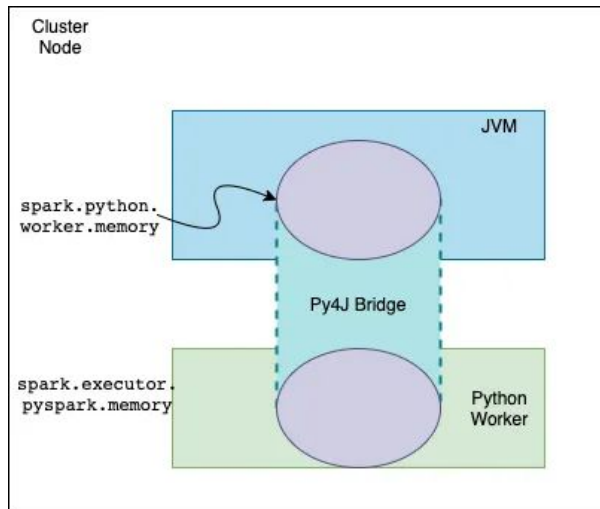
❖ Python 프로세스 + JVM 프로세스



실제 SparkContext는 JVM쪽에서
생성

◆ PySpark Memory (1)

- ❖ Spark은 JVM Application이지만 PySpark은 Python 프로세스
 - JVM에서 바로 동작하지 못함 따라서 JVM 메모리를 사용할 수 없음
- ❖ `spark.executor.pyspark.memory` (Python 프로세스)
- ❖ `spark.python.worker.memory` (Py4J)



Executor안에 2개의 프로세스가 존재

- JVM 프로세스
- Python Worker

이 둘간에 오브젝트
serialization/deserialization을
하는 것이 Py4J의 역할

◆ PySpark Memory (2)

❖ spark.executor.pyspark.memory

- PySpark은 기본으로 **overhead memory**를 사용. 이 환경변수가 사용되면 PySpark이 사용할 수 있는 메모리는 이 환경변수의 값으로 고정됨
- 이는 사실 PySpark이 외부 파이썬 함수를 쓰는 경우에만 필요 (기본적으로는 세팅되지 않음)

❖ spark.python.worker.memory

- 디폴트 값은 512MB (512m)
- JVM과 파이썬 프로세스간의 통신을 담당하는 **Py4J**가 사용할 수 있는 메모리의 양
- 이 크기를 넘어가면 디스크로 **Spill** 발생
- spark.executor.pyspark.memory는 파이썬 프로세스의 사용 메모리 크기 결정
- spark.python.worker.memory는 JVM에서 사용되는 파이썬 오브젝트들의 최대 메모리 결정

◆ Spark과 Python간의 통신

❖ Py4J: 파이썬과 JVM간의 데이터 교환을 통해 둘간의 연동을 도와주는 프레임워크

❖ DataFrame/RDD 연산중에 파이썬 코드가 사용되면?

- 이는 별도의 파이썬 프로세스를 통해 실행되며 이 경우 파티션 데이터가 모두 넘어감

```
df.select("foo", "bar").where(df["foo"] > 100).count()
```

vs.

```
from operator import add
```

```
t=spark.sparkContext.parallelize(range(len))
```

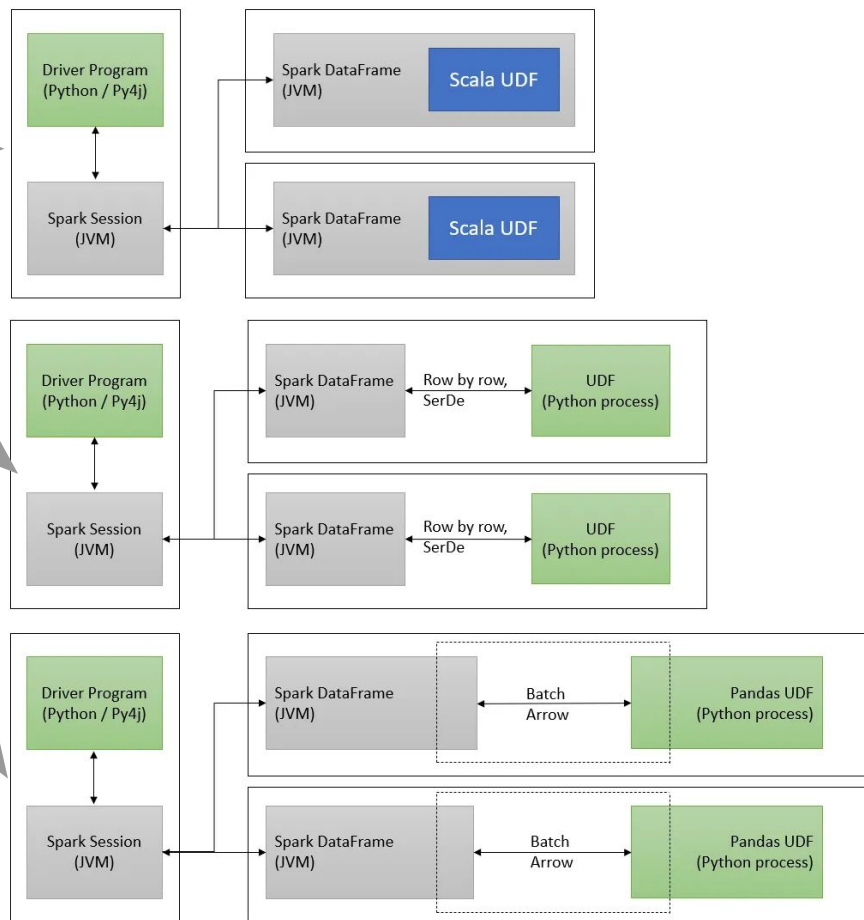
```
a = t.reduce(add)
```

- 드라이버에서 위의 파이썬 프로세스에서 실행할 코드와 기타 데이터를 **serialize**해서 **Executor**에 전송
- **JVM executor**는 파이썬 프로세스 실행 (**PySpark script**)
- **Executor**는 이것과 파티션을 **serialize**하고 위에서 받은 코드와 함께 파이썬 프로세스로 전송
- 파이썬 프로세스에서 계산이 끝나면 결과가 다시 **Executor**로 **serialize**되어서 전송됨

◆ Spark과 UDF

- ❖ Java나 Scala로 작성
- ❖ Python으로 작성
- ❖ Pandas Python으로 작성
 - Vectorized UDFs
 - PyArrow가 사용됨

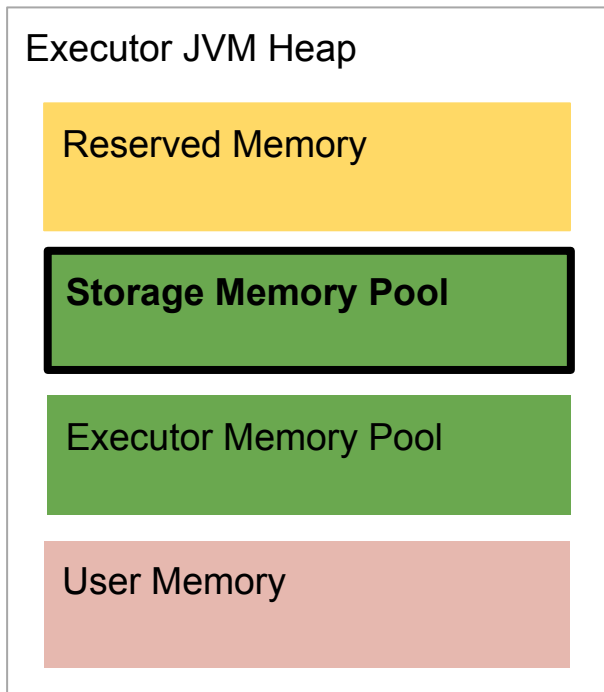
Batch Arrow로 넘어가는
레코드의 기본 수는 10K



Caching과 Persist

어느 데이터시스템이건 반복되어서 사용되는 데이터가 있다면 메모리에 두는 것이 좋은데 Spark에서 사용법에 대해 알아보자

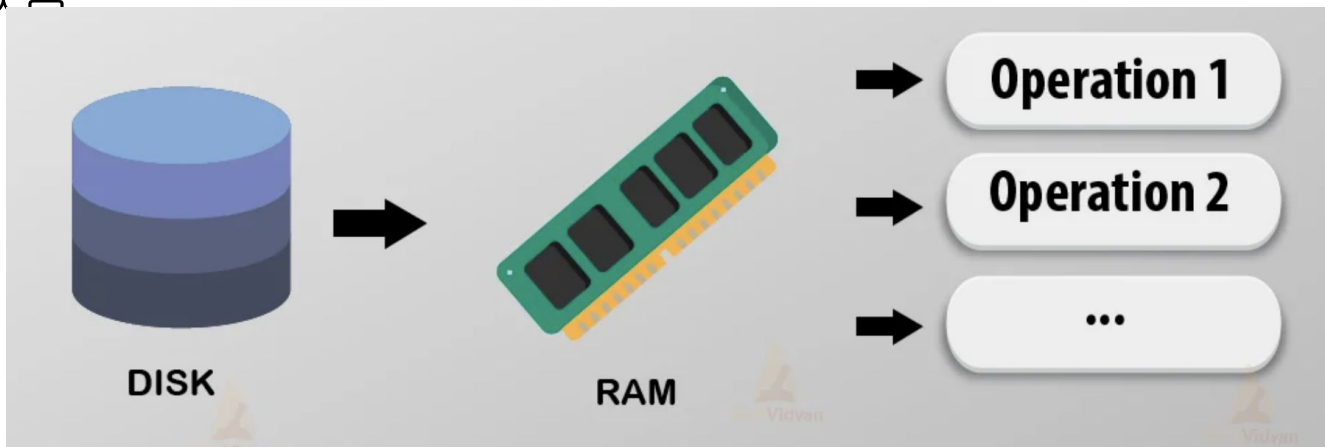
◆ Caching



1. Caching이란 무엇이며 왜 caching이 필요한가?
2. 어떻게 DataFrame을 caching하는가?
3. 언제 caching하고 언제 하지 말아야 하는가?
4. Caching을 취소하는 방법은?
5. Caching 포맷은?
6. Caching을 메모리에 할까? 디스크에 할까?

◆ Caching이란 무엇이며 왜 caching이 필요한가?

- ❖ 자주 사용되는 데이터프레임을 메모리에 유지하여 처리속도 증가
 - 단 그 데이터프레임이 정말 메모리에 있는지 확인 필요
 - 어떤 경우에는 다시 계산하는 것이 빠를 수도 있음
- ❖ 단 메모리 소비를 늘리므로 불필요하게 모든 걸 캐싱할 필요는 없음



Source: <https://techvidvan.com/>

◆ 어떻게 DataFrame을 caching하는가? (1)

❖ 두 가지 방법이 존재

- `cache()`
- `persist()`

❖ 둘다 데이터프레임을 메모리/디스크/오프heap에 보존

- 모두 **lazy execution** - 필요해지기 전까지 캐싱하지 않음
- **caching**은 항상 파티션 단위로 메모리에 보존
 - 하나의 파티션이 부분적으로 **caching**되지 않음

◆ 어떻게 DataFrame을 caching하는가? (2)

❖ persist는 인자를 통해 세부 제어 가능

- useDisk = True
- useMemory = True
- useOffHeap = False
 - off Heap 설정이 필요
- deserialized = False
 - 메모리를 줄일지 아니면 CPU 계산을 줄일지? 후자++
 - deserialized = True는 메모리에서만 가능
- replication = 1
 - 몇 개의 복사본을 서로 다른 executor에 저장할지 결정

◆ 어떻게 DataFrame을 caching하는가? (3)

❖ persist의 인자로 자주 사용되는 조합은 하나의 상수로 지정 가능

- DISK_ONLY
- MEMORY_ONLY
- MEMORY_AND_DISK
- MEMORY_ONLY_SER
- MEMORY_AND_DISK_SER
- OFF_HEAP
- MEMORY_ONLY_2
- MEMORY_ONLY_3

◆ 어떻게 DataFrame을 caching하는가? (4)

- ❖ persist는 기본적으로 caching되는 데이터프레임을 메모리와 디스크에 보관하고 복제도 수행함
- ❖ cache는 persist의 다음 버전
 - disk=False
 - memory=True
 - offHeap=False
 - deserialized=True
 - replication=1

◆ Spark SQL을 사용한 Caching

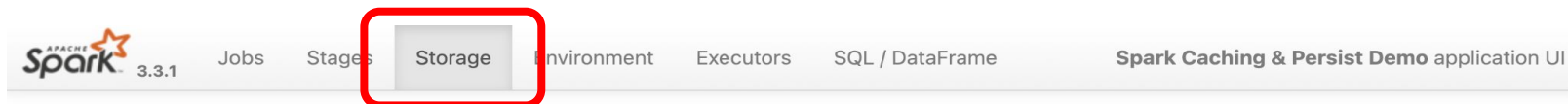
- ❖ `spark.sql("cache table table_name")`
- ❖ `spark.sql("cache lazy table table_name")`
- ❖ `spark.sql("uncache table table_name")`

- ◆ Caching을 취소하는 방법은?
 - ❖ DataFrame.unpersist (LRU - Least Recently Used)
 - ❖ spark.sql("uncache table table_name")
 - ❖ spark.catalog.isCached("table_name")
 - ❖ spark.catalog.clearCache()

◆ Caching 실습

❖ Caching 여부 확인

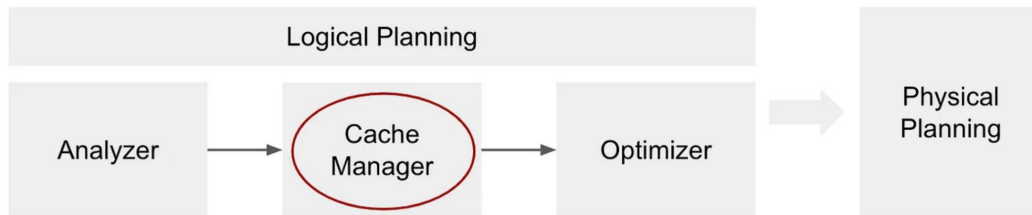
- Spark Web UI의 Storage 탭 확인



Storage

▼ RDDs

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
20	*(2) Project [id#0L, (id#0L * id#0L) AS square#9L] +- Exchange RoundRobinPartitioning(10), REPARTITION_BY_NUM, [plan_id=58] +- *(1) Range (1, 1000000, step=1, splits=3)	Disk Memory Deserialized 1x Replicated	1	10%	1566.3 KiB	0.0 B



◆ Caching 관련 Best Practices

- ❖ 캐싱된 데이터 프레임이 재사용되는 것을 분명하게 하기
 - `cachedDF = df.cache()`
 - `cachedDF.select(...)`
- ❖ 컬럼이 많다면 정말 필요한 컬럼만 캐싱
 - `cachedDF = df.select(col1, col2, col3, col4).cache()`
- ❖ 불필요할 때 **uncache**
- ❖ 때로는 매번 새로 데이터프레임을 계산하는 것이 캐싱보다 빠를 수 있음
 - 이는 큰 데이터셋이 **Parquet**와 같은 포맷으로 존재하는 경우
 - 캐싱 ❖ 소수의 데이터프레임만 캐싱[≠] 경우
 - ❖ 큰 데이터프레임의 캐싱은 하지 말것
 - ❖ 캐싱을 너무 믿지 말것

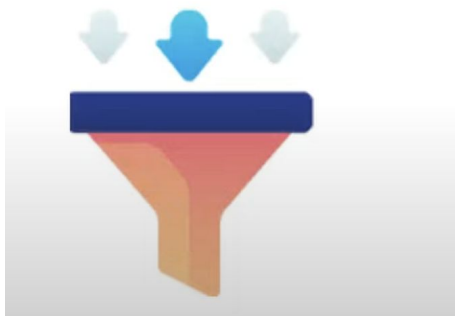
Dynamic Partition Pruning

처리 데이터가 **Partition**되어 있다면 이 특성을 이용해 처리 데이터를 줄이는 방법에 대해 살펴보자

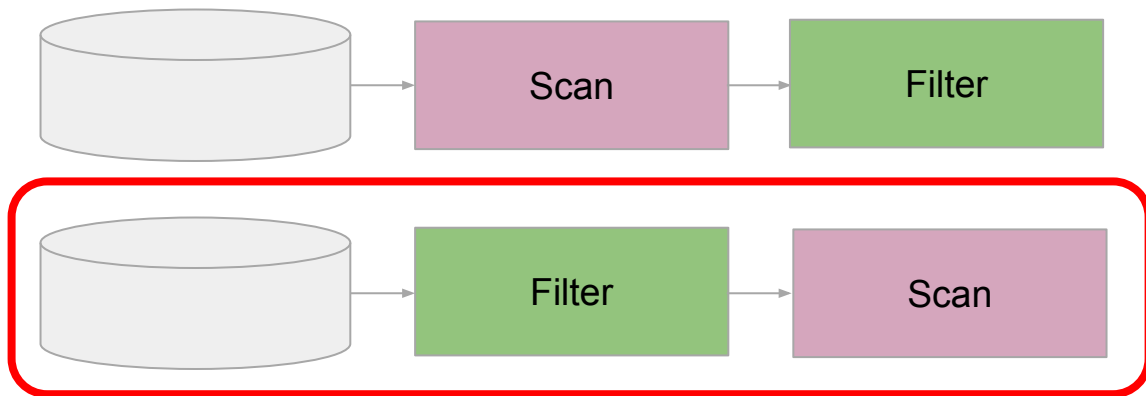
◆ Filter (Predicate) Pushdown

- ❖ 데이터 소스에서 읽어들이기 때 필터링을 적용해 읽는 데이터를 최소화
- ❖ 특정 데이터 소스에만 사용 가능 (PARQUET - 컬럼 통계 정보가 있는 경우)

Filter
pushdown



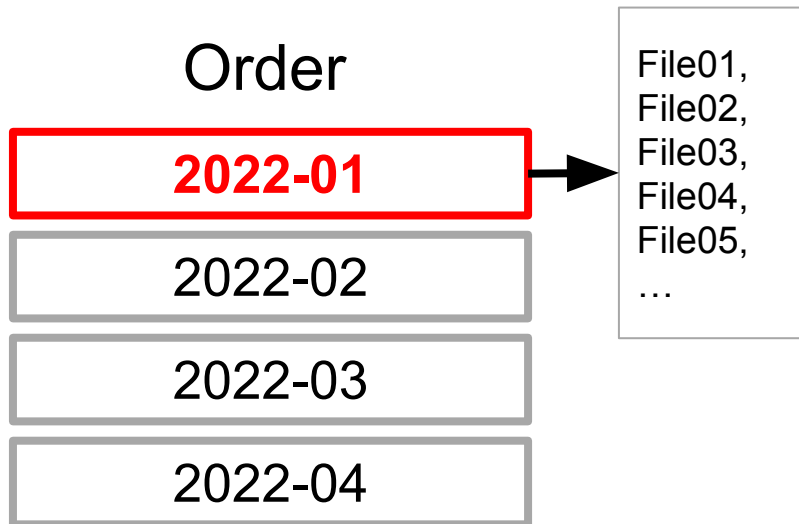
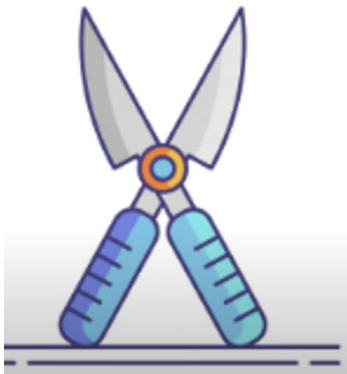
```
select * FROM order where order_month = '2022-01'
```



◆ Partition Pruning이란

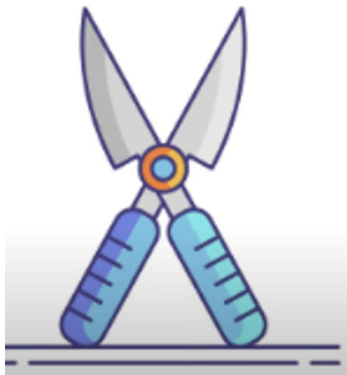
❖ 최적화 방식의 일종

- Optimizer가 정말 필요한 데이터와 불필요한 데이터를 구별하여 읽는 것!

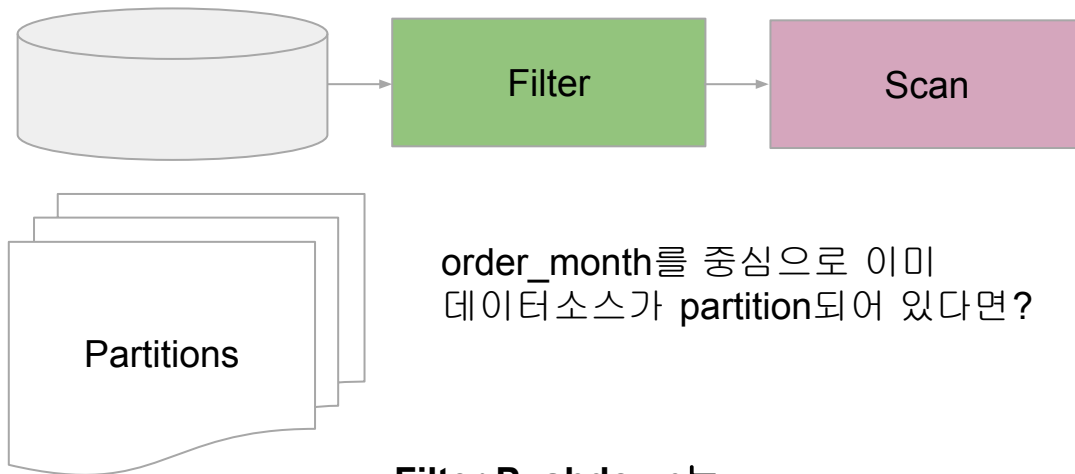


◆ Static Partition Pruning이란

❖ 데이터소스가 필터링 컬럼을 중심으로 Partitioning되어 있는 경우



```
select * FROM order where order_month = '2022-01'
```

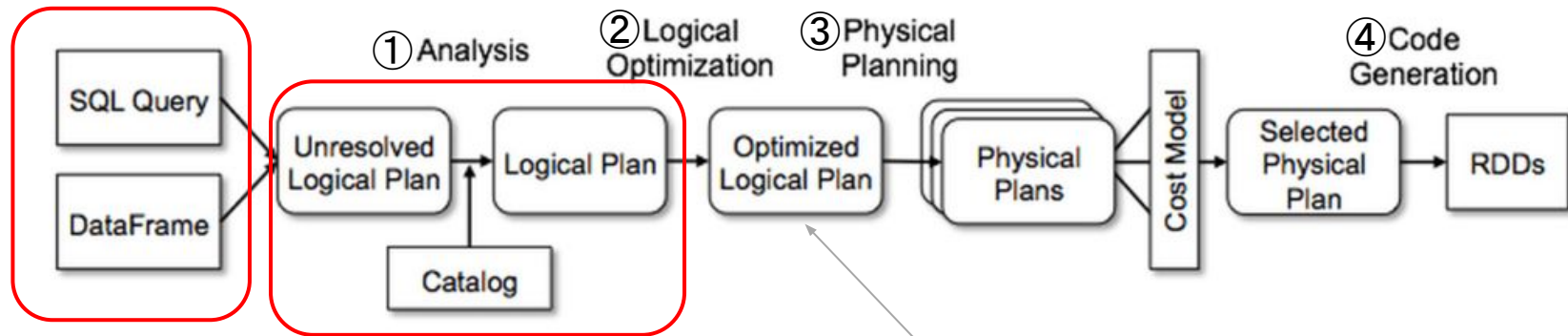


order_month를 중심으로 이미
데이터소스가 partition되어 있다면?

**Filter Pushdown는
Partition 테이블에서 더 효과적!**

◆ Partition Pruning과 Execution Plan

❖ Partition Pruning은 Logical Plan Optimization 단계에서 발생



constant folding
predicate pushdown
partition pruning
boolean expression simplification

```
int x = (2 + 3) * y →  
int x = 5 * y.
```

```
((A > 10) && (B < 20)) || ((A >  
20) && (B < 30)) -> A > 10) &&  
(B < 20)
```

◆ Static Partition Pruning의 문제

❖ Partitioning은 보통 큰 테이블에 적용되어 있음 (Fact 테이블)

item_id	price	order_date	sku	destination	factory	order_month
3331590	98.0	2022-11-01 00:00:00	PRODUCT-12	SEOUL	JEJU	2022-11
2316626	27.0	2022-11-04 00:00:00	PRODUCT-3	SEOUL	JEJU	2022-11
3331591	98.0	2022-11-01 00:00:00	PRODUCT-9	SEOUL	BUSAN	2022-11
2316627	nan	2022-11-30 00:00:00	PRODUCT-97	SEOUL	BUSAN	2022-11
3331592	22.0	2022-11-01 00:00:00	PRODUCT-5	SEOUL	JEJU	2022-11

order 테이블
(Partition
Fact)

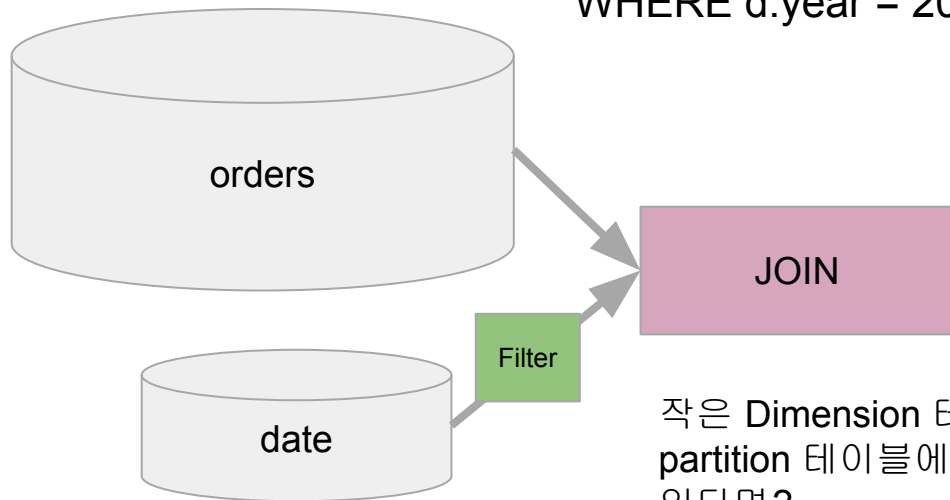
year	month	year_month
2016	1	2016-01
2016	2	2016-02
2016	3	2016-03
2016	4	2016-04
2016	5	2016-05

date 테이블 (No
Partition
Dimension)

◆ Static Partition Pruning의 문제

- ❖ Fact 테이블과 Dimension 테이블 조인시 필터링이 Dimension 테이블에 적용되어 있다면?

```
SELECT * FROM order o  
JOIN date d ON o.order_month = d.year_month  
WHERE d.year = 2022 and d.month = 1
```

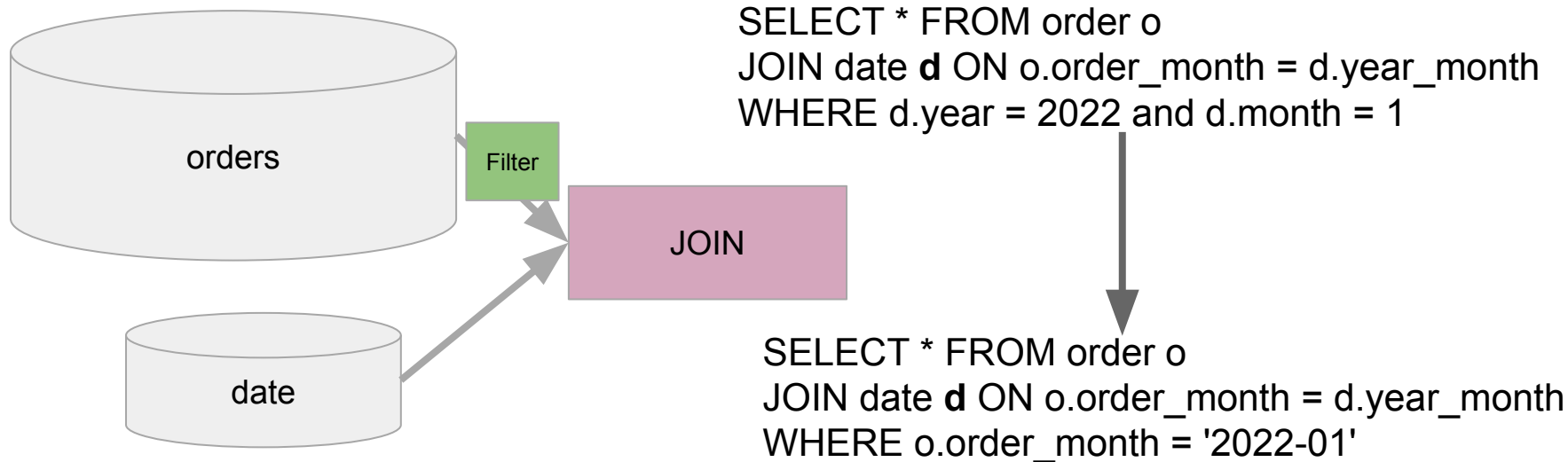


작은 Dimension 테이블에 적용된 필터링을
partition 테이블에 적용하여 읽기를 최소화할 수
있다면?

◆ Dynamic Partition Pruning이란?

❖ 비 Partition 테이블에 적용된 필터링을 Partition 테이블에 적용해보는 것

- 후자가 작은 dimension 테이블이라면 브로드캐스트 조인까지 하면 금상첨화



◆ Dynamic Partition Pruning 예제 보기

❖ 기본적으로 활성화되어 있음

- `spark.sql.optimizer.dynamicPartitionPruning.enabled: true`

❖ 예제 프로그램 보기

- `broadcast` 조인 확인해보기

`spark.sql.autoBroadcastJoinThreshold: 10MB`

요약

- 기타 기능/개념 살펴보기
- Driver와 Executor 해부
- 메모리 이슈 정리
- JVM과 Python 간의 통신
- Caching과 Persist
- Dynamic Partition Pruning



Grepp Inc

한기용

keeyonghan@hotmail.com